Cyberscope
A TAC Security Company

Audit Report
ESGIN Token

October 2026

Network	ETH
Address	0x9F293778c23DA45Eb7126DD4a8cB3923547e0C3D
Audited by	© cyberscope

Analysis

Critical Medium Minor / Informative Pass

Severity	Code	Description	Status
●	ST	Stops Transactions	Passed
●	OTUT	Transfers User's Tokens	Passed
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mints Tokens	Passed
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Passed

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
	CO	Code Optimization	Unresolved
	IDI	Immutable Declaration Improvement	Unresolved
	TSIC	Tokens Stuck In Contract	Unresolved
	ULS	Unverifiable Library Sources	Unresolved
	L04	Conformance to Solidity Naming Conventions	Unresolved
	L19	Stable Compiler Version	Unresolved

Table of Contents

Analysis 2
Diagnostics 3
Table of Contents 4
Risk Classification 5
Review 6
Audit Updates 6
Source Files 6
Findings Breakdown 7
CO - Code Optimization 8
Description 8
Recommendation 8
IDI - Immutable Declaration Improvement 9
Description 9
Recommendation 9
TSIC - Tokens Stuck In Contract 10
Description 10
Recommendation 10
ULS - Unverifiable Library Sources 11
Description 11
Recommendation 11
L04 - Conformance to Solidity Naming Conventions 12
Description 12
Recommendation 12
L19 - Stable Compiler Version 13
Description 13
Recommendation 13
Contract Functions 14
Inheritance Graph 15
Flow Graph 16

Summary 17
Disclaimer 18
About Cyberscope 19

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

> High criticality iteration: badge applicability has not been selected.

Contract Name	ESGIN
Compiler Version	v0.8.34+commit.80d5c536
Optimization	200
Explorer	https://etherscan.io/address/0x9F293778c23DA45Eb7126DD4a8cB3923547e0C3D
Address	0x9F293778c23DA45Eb7126DD4a8cB3923547e0C3D
Network	ETH
Symbol	ESGIN
Decimals	18
Total Supply	1,000,000,000

Audit Updates

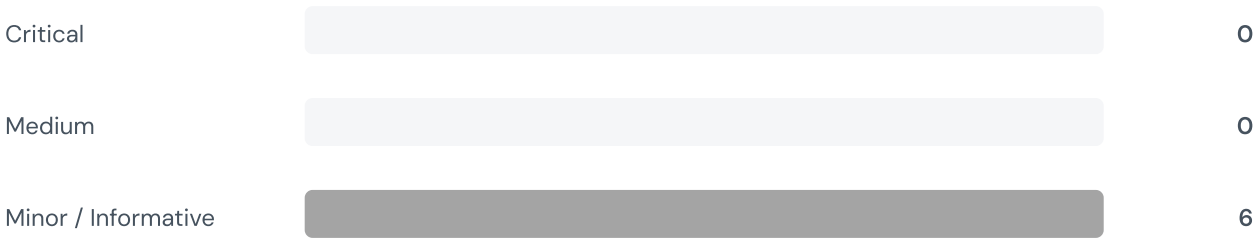
Initial Audit	20 Oct 2026

Source Files

Filename	SHA256
ESGIN.sol	2d96a53a8464fbb66935a40d6931b1fa7e01e4b29696886e270a7be90063eb7f

Findings Breakdown

6 Issues Found



Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	0	0	0	0
Minor / Informative	6	0	0	0

CO - Code Optimization

Severity	Location	Status
minor	ESGIN.sol#L233,402	unresolved

Description

There are code segments that could be optimized. A segment may be optimized so that it becomes a smaller size, consumes less memory, executes more rapidly, or performs fewer operations.

Specifically, locks with the same release time are stored as separate entries instead of sharing one, and the nonReentrant modifier guards functions that make no external calls.

SOLIDITY

```
modifier nonReentrant() {
...
locks.push();
```

Recommendation

The team is advised to take these segments into consideration and rewrite them so the runtime will be more performant. That way it will improve the efficiency and performance of the source code and reduce the cost of executing it.

IDI - Immutable Declaration Improvement

Severity	Location	Status
minor	ESGIN.sol#L297	unresolved

Description

The contract declares state variables that their value is initialized once in the constructor and are not modified afterwards. The `immutable` is a special declaration for this kind of state variables that saves gas when it is defined.

SOLIDITY

```
_totalSupply
```

Recommendation

By declaring a variable as immutable, the Solidity compiler is able to make certain optimizations. This can reduce the amount of storage and computation required by the contract, and make it more gas-efficient.

TSIC - Tokens Stuck In Contract

Severity	Location	Status
minor	ESGIN.sol#L354	unresolved

Description

Transfers to the token contract's own address are accepted, because `_transfer` only rejects the zero address. The contract never transfers or approves tokens from its own balance, so no path can move them out again. A user who sends tokens to the contract address by mistake loses them permanently.

SOLIDITY

```
require(to != address(0), "ERC20: transfer to zero");
```

Recommendation

The team is advised to prevent transfers whose recipient is the token contract itself.

ULS - Unverifiable Library Sources

Severity	Location	Status
minor	ESGIN.sol#L4,86,117,224	unresolved

Description

Library code from OpenZeppelin is pasted into the source with its versions recorded only in comments, and the project ships no dependency manifest. The ReentrancyGuard has no stated origin, and the file declares no SPDX license identifier. Reviewers cannot confirm that the library code matches an audited release or track upstream fixes.

SOLIDITY

```
// OpenZeppelin Contracts (last updated v5.0.0) (access/Ownable.sol)
```

Recommendation

The team is advised to source library code from a pinned, verifiable dependency and declare the license of the source file.

L04 - Conformance to Solidity Naming Conventions

Severity	Location	Status
minor	ESGIN.sol#L257,444,453,468	unresolved

Description

The Solidity style guide is a set of guidelines for writing clean and consistent Solidity code. Adhering to a style guide can help improve the readability and maintainability of the Solidity code, making it easier for others to understand and work with.

The followings are a few key points from the Solidity style guide:

1. Use camelCase for function and variable names, with the first letter in lowercase (e.g., myVariable, updateCounter).
2. Use PascalCase for contract, struct, and enum names, with the first letter in uppercase (e.g., MyContract, UserStruct, ErrorEnum).
3. Use uppercase for constant variables and enums (e.g., MAX_VALUE, ERROR_CODE).
4. Use indentation to improve readability and structure.
5. Use spaces between operators and after commas.
6. Use comments to explain the purpose and behavior of the code.
7. Keep lines short (around 120 characters) to improve readability.

SOLIDITY

```
uint8 private constant _decimals = 18
address _addr
uint256 _period
```

Recommendation

By following the Solidity naming convention guidelines, the codebase increased the readability, maintainability, and makes it easier to work with. Find more information on the Solidity documentation https://docs.soliditylang.org/en/stable/style-guide.html#naming-conventions.

L19 - Stable Compiler Version

Severity	Location	Status
minor	ESGIN.sol#L244	unresolved

Description

The `^` symbol indicates that any version of Solidity that is compatible with the specified version (i.e., any version that is a higher minor or patch version) can be used to compile the contract. The version lock is a mechanism that allows the author to specify a minimum version of the Solidity compiler that must be used to compile the contract code. This is useful because it ensures that the contract will be compiled using a version of the compiler that is known to be compatible with the code.

SOLIDITY

```
pragma solidity ^0.8.20;
```

Recommendation

The team is advised to lock the pragma to ensure the stability of the codebase. The locked pragma version ensures that the contract will not be deployed with an unexpected version. An unexpected version may produce vulnerabilities and undiscovered bugs. The compiler should be configured to the lowest version that provides all the required functionality for the codebase. As a result, the project will be compiled in a well-tested LTS (Long Term Support) environment.

Contract Functions

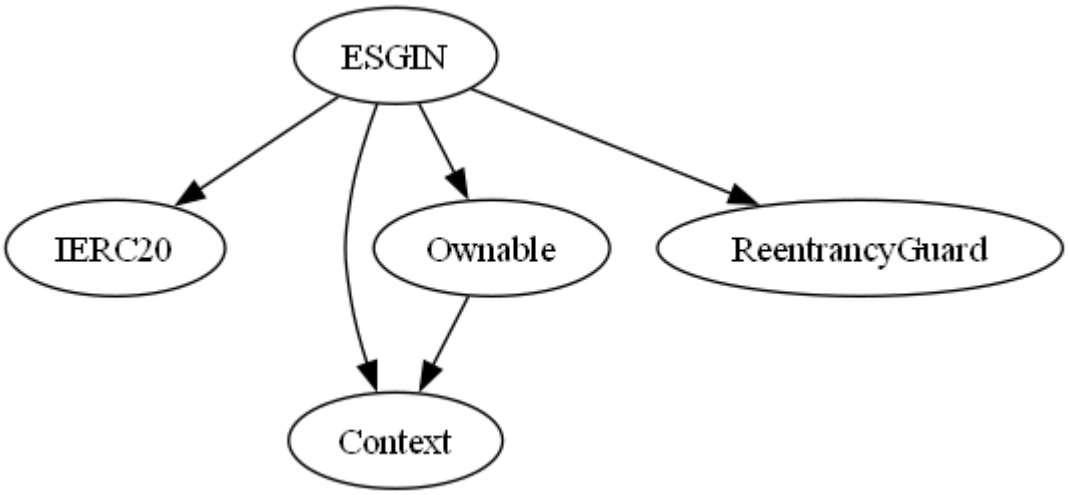
Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
IERC20	Interface			
	totalSupply	External		-
	balanceOf	External		-
	transfer	External	✓	-
	allowance	External		-
	approve	External	✓	-
	transferFrom	External	✓	-
Context	Implementation			
	_msgSender	Internal		
	_msgData	Internal		
	_contextSuffixLength	Internal		

Ownable	Implementation	Context		
	<Constructor>	Public	✓	-
	owner	Public		-
	_checkOwner	Internal		
	renounceOwnership	Public	✓	onlyOwner
	transferOwnership	Public	✓	onlyOwner
	_transferOwnership	Internal	✓	
ReentrancyGuard	Implementation			
	<Constructor>	Public	✓	-
ESGIN	Implementation	Context, IERC20, Ownable, ReentrancyGuard		
	<Constructor>	Public	✓	Ownable
	name	Public		-
	symbol	Public		-
	decimals	Public		-

	totalSupply	Public		-
	balanceOf	Public		-
	transfer	Public	✓	-
	transferFrom	Public	✓	-
	allowance	Public		-
	approve	Public	✓	-
	increaseAllowance	Public	✓	-
	decreaseAllowance	Public	✓	-
	_transfer	Internal	✓	
	transferWithLock	Public	✓	onlyApproved nonReentrant
	_lock	Internal	✓	
	_sortedInsert	Internal	✓	
	_removeLock	Internal	✓	
	_autoUnlock	Internal	✓	
	claim	External	✓	nonReentrant
	addApproved	External	✓	onlyOwner
	removeApproved	External	✓	onlyOwner
	setLockCooldownPeriod	External	✓	onlyOwner

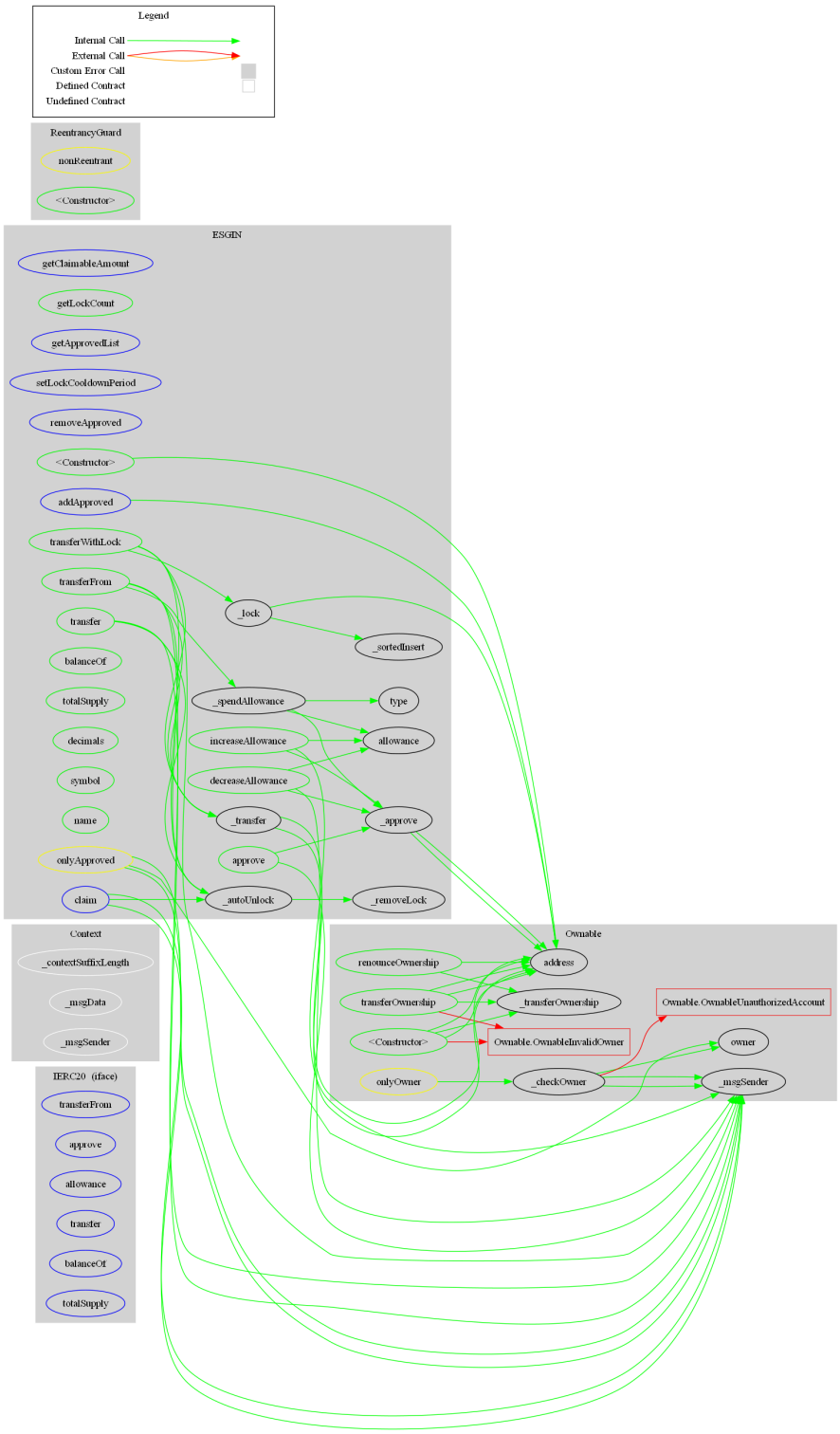
	getApprovedList	External		-
	getLockCount	Public		-
	getClaimableAmount	External		-
	_approve	Internal	✓	
	_spendAllowance	Internal	✓	

Inheritance Graph



Inheritance Graph

Flow Graph



Flow Graph

Summary

ESGIN contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an “endorsement” or “disapproval” of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



The Cyberscope team

cyberscope.io